Query Refinement for Diverse Top-k Selection

Reproducibility Package

This repository contains the ① reproducibility package for running our experiments and ② the implementation of our algorithm, which may be imported and used as a Python library.

If there are any issues evaluating the reproducibility package or otherwise using the implementation, please feel free to create an issue on GitHub or send an email.

① Reproducibility Package

❶ Install dependencies

- We containerized our experiments suite in order to ensure the correct dependencies are installed and the proper setup is performed.
  - The host machine should have 16 GB memory, and at least 256 GB of available disk space to ensure enough room to build the containers, hold the datasets, etc.
  - Please ensure Docker, docker-compose, and GNU make are all installed on your system, e.g. on Debian: `apt-get install docker docker-compose build-essential`

Important. In general, a license for **IBM CPLEX is required** for our implementation. CPLEX is available for free for academics (see `https://community.ibm.com/community/user/ai-datascience/blogs/xavier-nodet1/2020/07/09/cplex-free-for-students` for instructions on how to obtain such a license). Please ensure to place the `.bin` file of your licensed installer **(for Linux x86-64)** in the `docker-utils` folder, and the scripts will take care of the rest.

Note. For compatability, we recommend IBM ILOG CPLEX Optimization Studio V22.1.1 (part no. `G0798ML`). From it, download IBM ILOG CPLEX Optimization Studio V22.1.1 for Linux x86-64.

- Otherwise, no special configuration is required for the system – anything special is handled by our containers.

❷ Datasets

- Some of the datasets are too large to store in this repository. A link is available in the associated files, or please contact us.
- Unzip the data archive file `data.zip`, and place the `data` directory in the `ranking_refinements` directory
  - *Note*: Some of the datasets included in the data archive file are tracked in the repository – overwriting the contents of the `data` directory with the one extracted from `data.zip` is okay.

❸ Running experiments, generating figures, & recompiling the paper

- Please ensure that the **Docker daemon is running** on the host machine before proceeding.
- In order to run the experiments, generate plots, and recompile the paper, simply run (from this directory): `make paper`
- We expect that the full suite of experiments will take ~3-5 days depending on hardware setup.

- The script runs *all* experiments from which *all data presented in each graph shown in the paper* was collected.
- The script will also automatically *generate new plots* from the collected data, and *recompile the paper* with the new figures included. If the script terminates successfully, the paper `main.pdf` should be placed into this directory.
- If something goes wrong, please try `make clean && make paper` or feel free to contact us.

Appendix

Modifying experiments

- All experiments are stored as configuration files in `ranking_refinements/experiments_conf`.
- The queries, constraints, maximum average deviations ε, methods/algorithms, k^*, and other parameters may be controlled by these configuration files.
- Adding a new experiment configuration file requires modifying `ranking_refinements/scripts/run_experiments.sh` in order for it to be run with the suite.

Running a single set of experiments

- It is possible to run one set of experiments instead of running the whole suite.
  - This is helpful in case any changes are made to the experiment configuration files, or in the case that an experiment needs to be rerun for other reasons (e.g. an unexpected runtime issue).
- Running `make help` from this directory will show the sets of experiments that are available to run individually.
- After running any experiment, running `make fix-paper` will recompile the paper without running the *full* suite of experiments.

Clean-up

- In the case you would like to clean everything up (e.g. starting a fresh run), simply run: `make clean`

---

② Algorithm Implementation

Structure

- The technical details behind our algorithm may be found in our technical report, available at `http://arxiv.org/abs/2403.17786`
- All of the source code for implementation is in `ranking_refinements` folder
  - `ranking_refinements/fair.py` contains the key components of the implementation, and some example scenarios that may be run by evaluating the file

Getting everything set up

- We assume in this section that you'd like to use the library *outside* of the provided Docker containers for the evaluation of our experiments suite.
- Please ensure CPLEX is installed, as well as its included Python bindings.
  - See `https://www.ibm.com/docs/en/icos/20.1.0?topic=cplex-installing` for instructions on installing CPLEX, and `https://www.ibm.com/docs/en/icos/20.1.0?topic=cplex-setting-up-python-api` for instructions on setting up the Python bindings
- From this directory, run `pip install .`
  - This should install all the necessary dependencies and allow you to use the library.

Running the algorithm

- At the end of `ranking_refinements/fair.py`, there are a few example cases with various datasets
- Let's look at one example scenario in order to illustrate the usage of our library:

```
from ranking_refinements.fair import Ranking, Constraints, Constraint, Group,
↪  UsefulMethod, RefinementMethod

constraints = Constraints(
    Constraint(Group("Gender", "F"), (4, 2)),
    Constraint(Group("Gender", "M"), (4, 2)),
)

ranking = Ranking('SELECT * FROM "data/candidates.csv" WHERE ("Major" = \'CS\' OR
↪  Major = \'EE\') AND "Hours" >= 90 AND "Hours" <= 100 ORDER BY "Gpa" DESC')

refinement, times = ranking.refine(constraints, max_deviation=0, debug=True,
↪  method=RefinementMethod.MILP_OPT, useful_method=UsefulMethod.QUERY_DISTANCE)
print(refinement.conditions)
```

Tip. The quickest way to try refining a query is to edit one of the scenarios included at the bottom of `ranking_refinements/fair.py`, and then running `python fair.py` from inside the `ranking_refinements` directory.

Constraints

- `Constraints` holds many instances of `Constraint`
  - An instance of `Constraint` is initialized with a `Group` and a tuple `(k, cardinality)`
  - A `Group` is initialized with an attribute from the database, and a value in its domain
  - For example, `Constraint(Group("Gender", "F"), (4, 2))` encodes the constraint: *At least 2 of the top-4 candidates should belong to the group* `Gender=F`
  - When initializing `Constraint`, the optional `sense` keyword argument can be `L` or `U` for at-least and at-most respectively.

Rankings

- `Ranking` holds an SPJ query with (1) a **`WHERE`** clause and (2) an **`ORDER BY`** clause.
  - An instance of `Ranking` is initialized with an SQL query (as a string)

Refinements

- With a `Ranking` and a set of `Constraints`, we may refine the query by calling the `refine` method on the `Ranking` object
  - For our example, we would call `ranking.refine(constraints)`
  - `refine` can be configured with many parameters, such as
    - `max_deviation`: Maximum average deviation allowed from the constraint set
    - `useful_method`: One of `UsefulMethod.QUERY_DISTANCE, UsefulMethod.MAX_ORIGINAL, UsefulMethod.KENDALL_DISTANCE`
    - `method`: One of `RefinementMethod.MILP_OPT, RefinementMethod.MILP, RefinementMethod.BRUTE_PROV, RefinementMethod.BRUTE`